

Lecture 3: Support Vector Machines and Unsupervised Learning

Alex Reinhart
Machine Learning II: 46-927

February 8–10, 2021

Announcements

- ▶ Currently reviewing project ideas
- ▶ Check the project description on Canvas
- ▶ Proposals (1 paragraph) due Friday, February 19th
- ▶ Quiz 1 returned; currently grading Homework 1

Today

Recently we discussed classification: tools like LDA and QDA, naive Bayes, and the difference between discriminative and generative classifiers. Then you got into trees: decision trees, random forests...

- ▶ Another discriminative classifier: Support vector machines
 - ▶ Maximum margin classifiers
 - ▶ Support vector classifiers
 - ▶ Kernel trick
- ▶ Unsupervised learning: Returning to Clustering and Dimension reduction in more detail.
 - ▶ Clustering
 - ▶ Dimension reduction? (likely next time)

Building a classifier

“We” showed that the ideal classifier is the Bayes classifier:

$$\begin{aligned}\hat{f}(x) &= \operatorname{argmax}_{k=1,\dots,K} P(Y = k \mid X = x) \\ &= \operatorname{argmax}_{k=1,\dots,K} P(X = x \mid Y = k) \cdot \pi_k\end{aligned}$$

To build a classifier, we either:

1. Estimate $P(Y = k \mid X = x)$ directly. (Discriminative models)
2. Estimate both π_k and $P(X = x \mid Y = k)$ for each class k .
(Generative models)

Discriminative models

Discriminative models are nice because $P(Y = k \mid X = x)$ is often much simpler to model:

- ▶ When $X \in \mathbb{R}^p$, p large, estimating $P(X = x \mid Y = k)$ is challenging (high-dimensional density estimation)
- ▶ Can adapt many tools to estimate $P(Y = k \mid X = x)$

First: hyperplanes

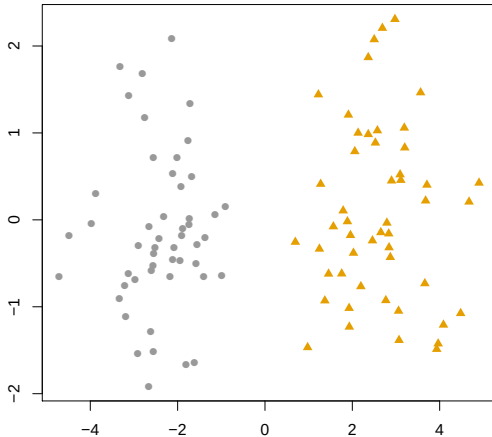
Several discriminative classifiers look at $\beta_0 + x_i^T \beta$, and assign

$$\begin{aligned}\hat{f}(x) &= \begin{cases} 1 & \hat{\beta}_0 + x^T \hat{\beta} > 0 \\ -1 & \hat{\beta}_0 + x^T \hat{\beta} < 0 \end{cases} \\ &= \text{sign} \left(\hat{\beta}_0 + x^T \hat{\beta} \right),\end{aligned}$$

where $x \in \mathbb{R}^p$, $\beta \in \mathbb{R}^p$, $\beta_0 \in \mathbb{R}$, and $y \in \{-1, 1\}$. How should we think about this geometrically?

Hyperplane geometry

With $X \in \mathbb{R}^2$:



Building a linear classifier

Suppose I want to build a nice, linear classifier $\text{sign}(\beta_0 + x^T \beta)$.
How should I choose β and β_0 ?

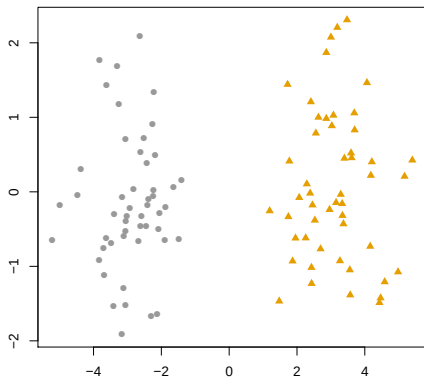
1. I could build a model of each cloud of points, and classify to the best model
2. I could model probability $P(Y = 1 \mid X)$ with a linear model,

$$P(Y = 1 \mid X = x) = \frac{1}{1 + e^{-(\beta_0 + x^T \beta)}}$$

3. I could just try to draw a line down the middle.

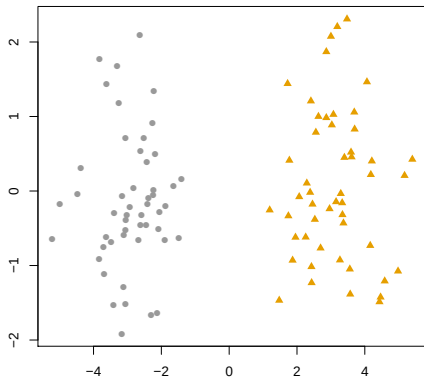
Maximum margin classifier: a special case

Where should we put our separating plane?



Maximum margin classifier: a special case

Where should we put our separating plane?



Maximum margin classifier: a special case

Maximize M

Subject to $y_i(\beta_0 + x^T \beta) \geq M$ and $\|\beta\|_2^2 = 1$

by choosing β, β_0, M

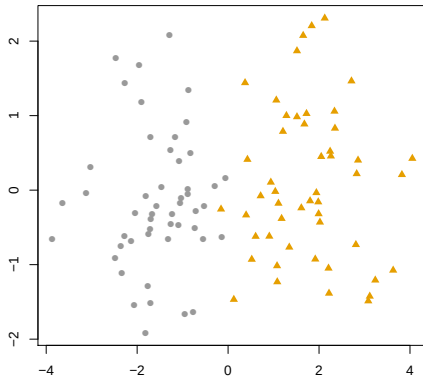
This is not based on model assumptions! This is just a nice idea of what “draw a separating line” should look like.

Note that the plane only depends on the points right at the boundary. The other points could move around and nothing would change.

However, this is only defined if we have nicely separable groups! That seems a bit wishful.

Maximum margin classifier: a special case

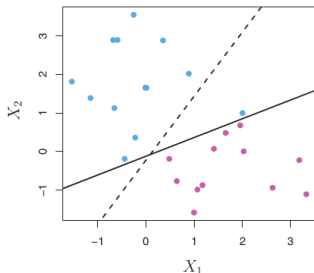
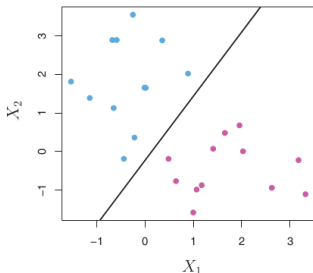
Now what??



Support vector classifier

We need to relax the notion of a margin, in case the groups cannot actually be separated. We introduce a notion of a **soft margin** which allows some violations.

This has unexpected benefits! Strict margins give the boundary points too much influence. Now we can tune the variance of the boundary.



(ISL pg.345)

Support vector classifier

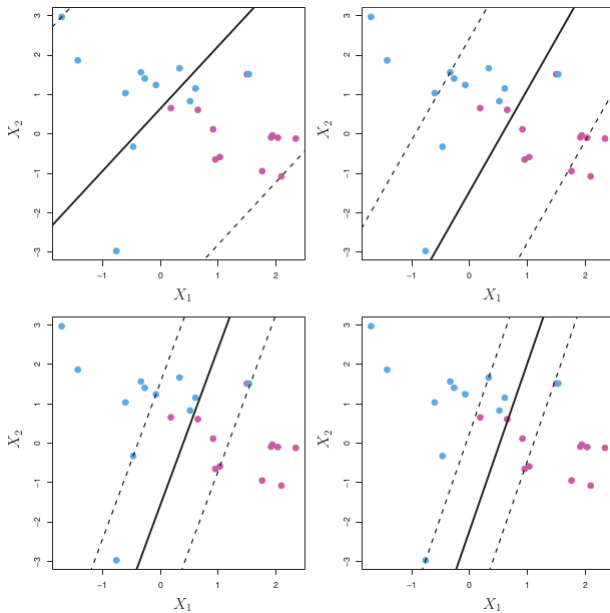
Maximize M

subject to $\|\beta\|_2^2 = 1, \varepsilon_i \geq 0, \sum_{i=1}^n \varepsilon_i \leq C$

$$y_i(\beta_0 + x_i^T \beta) \geq M(1 - \varepsilon_i)$$

by choosing $\beta_0, \beta, M, \epsilon$

- ▶ Tuning parameter C determines “softness” of the margin. Big C makes it easier to cross. In particular, no more than C observations cross because...
- ▶ Variable ε_i encodes point location: $\varepsilon_i = 0$ outside margin, $\varepsilon_i > 0$ inside margin, $\varepsilon_i > 1$ across boundary.



(ISL pg. 348)

Tuning the classifier

The parameter C trades off bias and variance.

It essentially determines how far from the boundary we should be looking when forming our line.

We can tune C using cross-validation.

We now have yet another linear classifier. This one requires even fewer assumptions, but still allows tuning and gives reasonable answers.

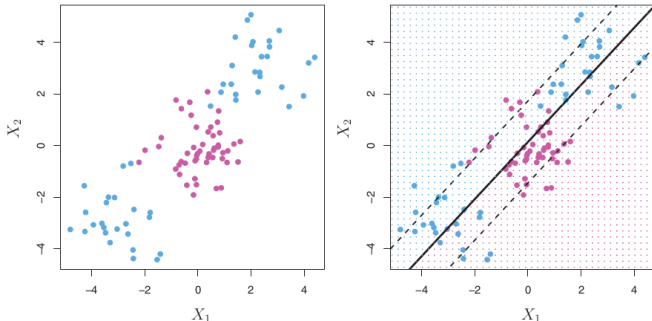
How do these linear classifiers compare?

Though the decision rules are all linear, these methods produce different classifications because they rely on different assumptions.

In order from strongest to weakest assumptions:

1. LDA: Works well if the groups are in “clumps” so that the Gaussian distribution is reasonable. Also assumes the shapes are similar.
2. Logistic: Only requires that $\log \frac{P(Y=1|X=x)}{P(Y=0|X=x)} \approx \beta_0 + x_i^T \beta$, which is strictly weaker. LDA will do better when its assumptions are reasonable, but otherwise worse. Logistic focuses more on the boundary points.
3. SVM: The fewest assumptions, and entirely focuses on points near the boundary. Can work well with the other models are far from reasonable. Also most common to use kernels in this one.

Support Vector Classifiers give linear boundaries. What are we going to do if we want something non-linear?



(ISL pg.349)

We could start making up transformations of the x values. For example, polynomials in the x , like x_1^2, x_2^2, x_1x_2 .

Linear boundaries in this higher dimensional space of transformed x values are equivalent to non-linear boundaries in lower dimensional space. You've seen this before in regression when adding terms like quadratics.

In high dimensions, this can be a lot to keep track of and to compute. Even constructing reasonable expansions can be difficult.

The “kernel trick” gives a convenient way around this.

Transformation demo

[Nonlinear SVM example.ipynb]

We can show (though we will not; see ESL 12.3) that the SVM really only depends on the inner products between pairs of observations x_i and $x_{i'}$.

Suppose we knew a nice, high dimensional transformation $\phi : \mathbb{R}^n \rightarrow \mathbb{R}^m$, and we wanted to fit a linear SVM in that $\mathbb{R}^m$ space to get better separation.

We would still only need the inner products in that higher dimensional space!

What if we cheat? Can we just compute the inner products directly, without ever forming the high-dimensional vectors?

We define a **kernel** function, which measures the similarity of two observations. For example:

Linear kernel: $K(x_i, x_{i'}) = \sum_{j=1}^p x_{ij}x_{i'j}$ is our usual inner product

Polynomial kernel: $K(x_i, x_{i'}) = \left(1 + \sum_{j=1}^p x_{ij}x_{i'j}\right)^d$

Radial kernel: $K(x_i, x_{i'}) = \exp\left(-\gamma \sum_{j=1}^p (x_{ij} - x_{i'j})^2\right)$

All of these correspond to forming higher dimensional vectors by transforming x , and then taking usual inner products. However, we never need to actually form the vectors!

Example: Polynomial kernel of order 2

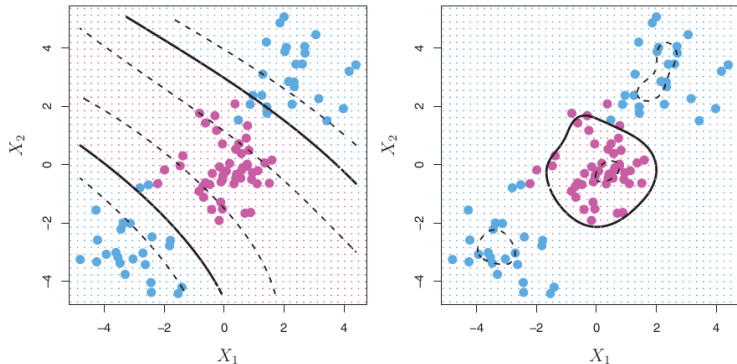
Suppose we observe points (x_1, x_2) , but we want to work in the space

$$(1, \sqrt{2}x_1, \sqrt{2}x_2, \sqrt{2}x_1x_2, x_1^2, x_2^2),$$

so that we can have quadratic boundaries. (Constants are just chosen to make the math nice)

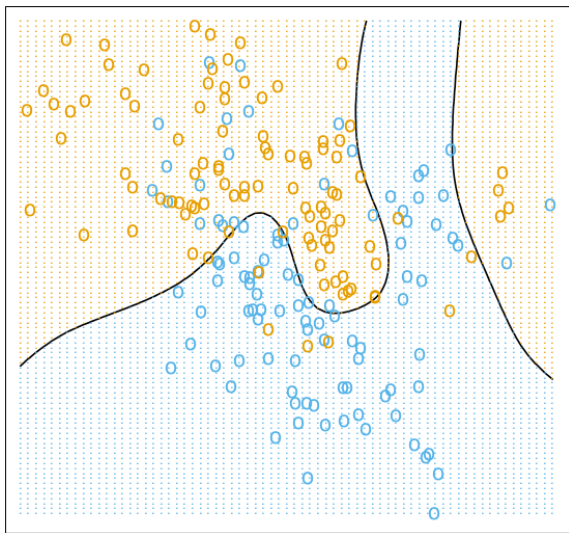
Compare taking inner products in the larger space to the kernel $K(x, y) = (1 + \sum_{i=1}^p x_i y_i)^2$

We can look for good linear boundaries in the high dimensional space (with no new math!), which will correspond to non-linear boundaries in our new space.



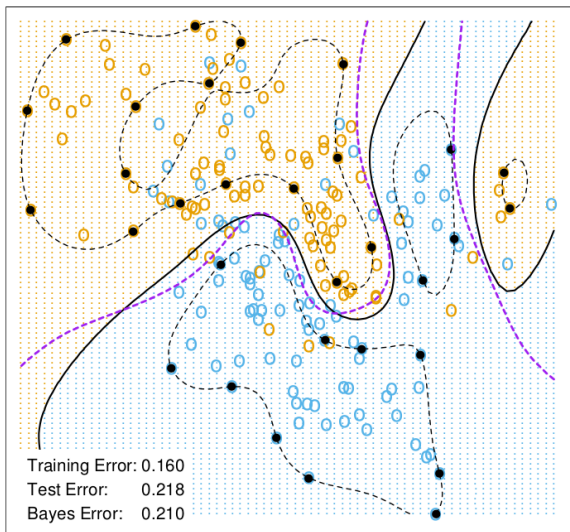
(ISL pg. 353)

The end result can be quite flexible



(ESL pg. 21)

The end result can be quite flexible



(ESL pg. 425)

Kernel demo

[SVM example.ipynb]

Empirical risk minimization

This general pattern:

1. We want to minimize:

$$\mathbb{E}\mathbf{1}\{Y \neq f(X)\}$$

2. And so we actually try to minimize its empirical version:

$$\frac{1}{n} \sum_{i=1}^n \mathbf{1}\{y_i \neq f(x_i)\}$$

3. But we can't even do that for classification. So we introduce a nicer loss L and minimize

$$\frac{1}{n} \sum_{i=1}^n L(y_i, f(x_i))$$

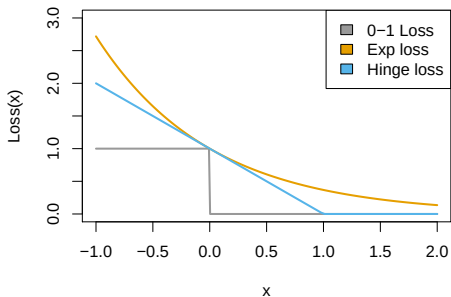
The first two steps are known as *empirical risk minimization*. The last step almost always follows for classification.

SVM: a different perspective

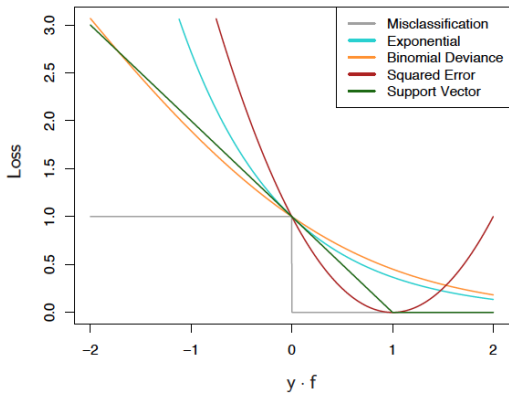
It might seem like SVM has no $L(y_i, f(x_i))$, just a crayon and a ruler. However, we can think about it another way:

$$\min_{\beta, \beta_0} \sum_{i=1}^n \left(1 - y_i(x_i^T \beta + \beta_0)\right)_+ + \frac{\lambda}{2} \|\beta\|_2^2$$

This is like replacing 0–1 loss with a hinge function and adding a ridge penalty to keep things regularized!



More losses



Something completely different

One of the things we saw from LDA is that understanding how the method *works* gives insights that can be leveraged into new methodology.

- ▶ In that case: supervised dimension reduction that emphasizes class differences.

This is a common trend. We'll continue bringing this up throughout class.

For today, we'll consider the case of **anomaly detection**

Anomaly detection

Suppose that you are observing a sequence of observations $X_1, X_2, \dots$ that are supposed to be i.i.d. from some distribution $\mathcal{P}$.

Anomaly detection is the problem of recognizing X_i that look “unusual”.

(There are many versions of this problem, including time series settings where you look for larger scale patterns. We'll just consider the i.i.d. case.)

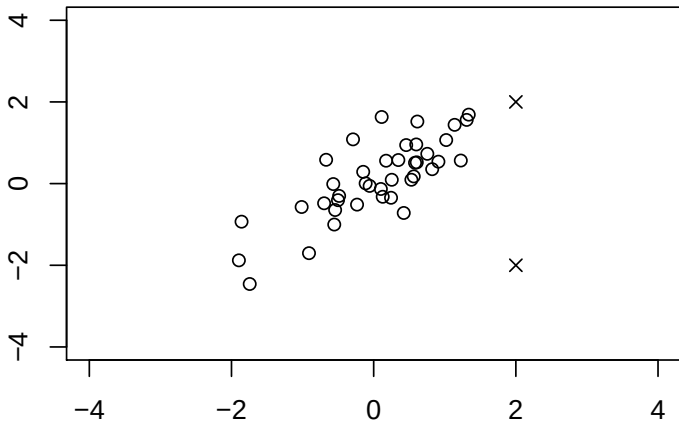
Anomaly detection

We observe $X_1, X_2, \dots \stackrel{iid}{\sim} \mathcal{P}$. We want to recognize *unusual* points, without flagging points drawn from $\mathcal{P}$ too often.

If we don't know what the unusual points will look like, one approach is just to look for points that are rare under $\mathcal{P}$.

Example: What if $\mathcal{P}$ is just $N(\mu, \sigma^2)$?

Anomaly detection: Gaussian case



Anomaly detection: Gaussian case

What if we're observing higher dimensional $X_i \in \mathbb{R}^p$?

If we think $\mathcal{P}$ looks like a multivariate Gaussian, we can use our knowledge there.

$$f(x; \mu, \Sigma) = \frac{1}{\sqrt{2\pi}|\Sigma|} e^{-\frac{1}{2}(x-\mu)^T \Sigma^{-1}(x-\mu)}$$

The density is directly a transformation of the Mahalanobis distance! Rare points have large

$$(x - \mu)^T \Sigma^{-1} (x - \mu)$$

Anomaly detection: Unknown distribution

What if we don't think we know $\mathcal{P}$, but doubt that it's Gaussian?

Many approaches exist. One is the **one-class SVM**.

Another is actually based on random forests! (No time today)

One-class SVM

In the usual SVM, we seek to separate two classes of points with a “plane”. We solve the following optimization:

$$\text{Minimize } \|\beta\|^2 + D \sum_{i=1}^n \varepsilon_i$$

$$\text{Subject to } y_i(x_i^T \beta + \beta_0) \geq (1 - \varepsilon_i), \varepsilon_i \geq 0$$

In the one-class optimization, we only have one group of points. Instead, we seek to separate the points from the origin.

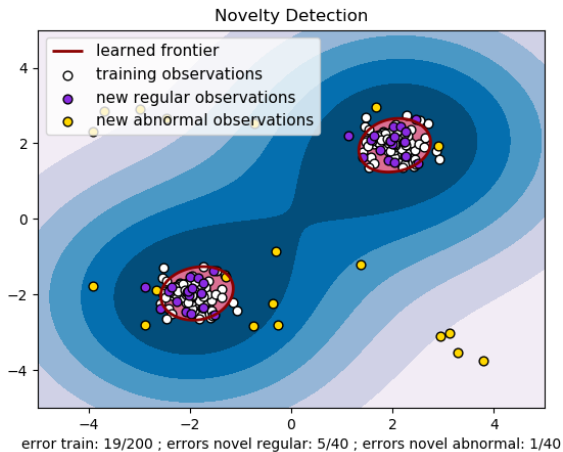
$$\text{Minimize } \|\beta\|^2 + \frac{1}{n\nu} \sum_{i=1}^n \varepsilon_i - \rho$$

$$\text{Subject to } (x_i^T \beta) \geq \rho - \varepsilon_i, \varepsilon_i \geq 0$$

One-class SVM

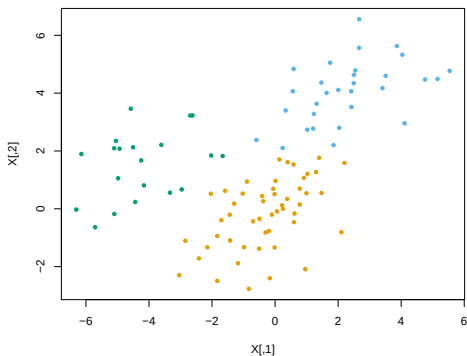
What does this do?

One-class SVM

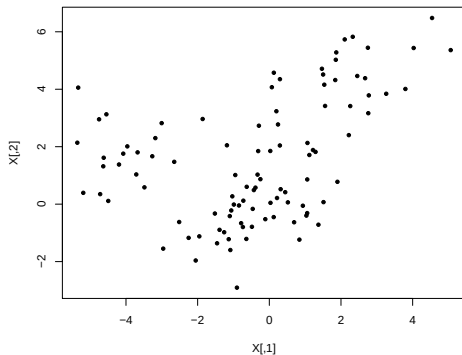


(From sklearn documentation)

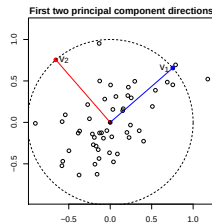
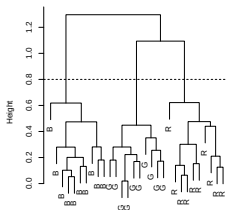
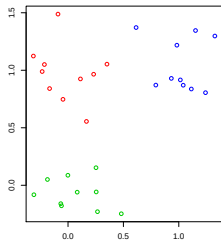
Unsupervised learning



So far, we have been looking at **labeled** data, where we observe variables $X_1, \dots, X_p$ and an outcome Y . In that setting (supervised learning), we try to predict Y for new variables $X_1, \dots, X_p$.



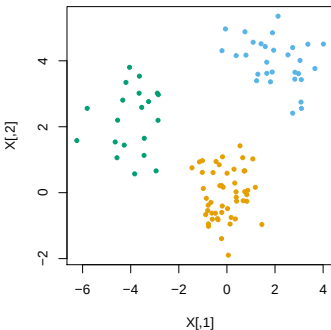
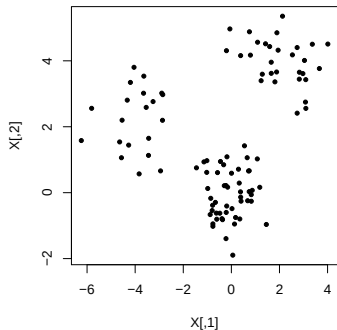
Sometimes we don't have an observed outcome Y , but we still want to understand something about the structure of our variables $X_1, \dots, X_p$. Questions of this sort are often referred to as **unsupervised learning**.



Exploring the structure in X includes methods like **clustering**, **dimension reduction** (e.g. PCA).

What is clustering? And why?

Clustering: task of dividing up data into groups (clusters), so that points in any one group are more “similar” to each other than to points outside the group



Why cluster?

Why cluster? Two main uses

- ▶ **Summary**: deriving a reduced representation of the full data set.
- ▶ **Discovery**: looking for new insights into the structure of the data. E.g., finding groups of students that commit similar mistakes, groups of clients with similar behaviors, groups of assets with high dependence, or groups of users with similar behaviors/likes/clicks.

Other uses, e.g.,

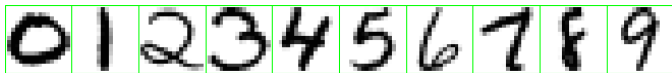
- ▶ **Helping with prediction**, i.e., in classification or regression

In finance

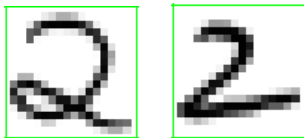
- ▶ Find groups of highly correlated assets for arbitrage strategies.
- ▶ Identify unrelated assets for portfolio diversification purposes
- ▶ Segment customers or other individuals for analysis, targeting, marketing, trading behavior
- ▶ Summarize data, create features

Don't confuse clustering and classification!

In classification, we have data for which the groups are **known**, and we try to learn what differentiates these groups (i.e., classification function) to properly classify future data



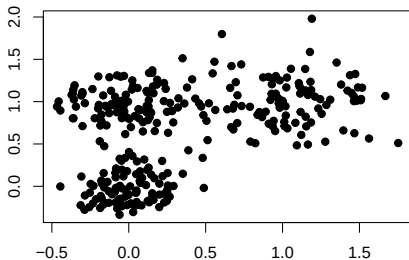
In clustering, we look at data for which groups are **unknown and undefined**, and try to learn the groups themselves, as well as what differentiates them



Some terminology and notation

Given observations $X_1, \dots, X_n$, and **dissimilarities** $d(X_i, X_j)$. (E.g., think of $X_i \in \mathbb{R}^p$ and $d(X_i, X_j) = \|X_i - X_j\|_2^2$)

Let K be the **number of clusters** (fixed). A **clustering** of points $X_1, \dots, X_n$ is a function C that assigns each observation X_i to a group $k \in \{1, \dots, K\}$



What makes a good cluster?

In supervised learning, we had a very good idea what made a good prediction function: Loss functions, misclassification rates, actual costs, etc.

What makes a good clustering?

- ▶ Tightly packed groups?
- ▶ Well-separated groups?
- ▶ What does it even mean for points to be “close”?

You'll eventually find that a clustering is “good” if it turns out to be *useful*, usually for some downstream purpose.

We'll explore several versions of all of these notions, each of which will be useful...sometimes.

Within-cluster scatter

One measure of a clustering is **within-cluster scatter**. This is a measure of how spread out the points are within each cluster.

The general notion is that a good clustering should lead to tightly-packed clusters with low within-cluster scatter.

Notation: $C(i) = k$ means that X_i is assigned to group k , and n_k is the number of points in the group k . Also, let $d_{ij} = d(X_i, X_j)$

The **within-cluster scatter** is defined as

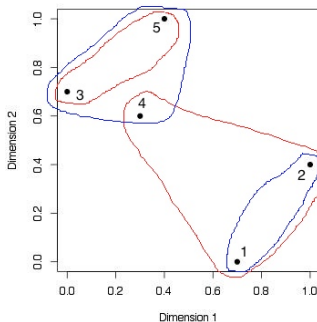
$$W = \frac{1}{2} \sum_{k=1}^K \frac{1}{n_k} \sum_{C(i)=k} \sum_{C(j)=k} d_{ij}$$

Smaller W is better

Simple example

Here $n = 5$ and $K = 2$,
 $X_i \in \mathbb{R}^2$ and $d_{ij} = \|X_i - X_j\|_2^2$

	1	2	3	4	5
1	0	0.25	0.98	0.52	1.09
2	0.25	0	1.09	0.53	0.72
3	0.98	1.09	0	0.10	0.25
4	0.52	0.53	0.10	0	0.17
5	1.09	0.72	0.25	0.17	0



► **Red clustering:**

$$W_{\text{red}} = (0.25 + 0.53 + 0.52)/3 + 0.25/2 = 0.56$$

► **Blue clustering:**

$$W_{\text{blue}} = 0.25/2 + (0.10 + 0.17 + 0.25)/3 = 0.30$$

Finding the best group assignments

Smaller W is better, so why don't we just directly find the clustering C that **minimizes W** ?

Problem: doing so requires trying **all possible assignments** of the n points into K groups. The number of possible assignments is huge!

For 25 points and 4 groups: $\approx 5 \times 10^{13}$

Most problems we look at are going to have way more than $n = 25$ observations, and potentially more than $K = 4$ clusters too.

So we'll have to settle for an **approximation**

Rewriting the within-cluster scatter

Focus on Euclidean space: now $X_i \in \mathbb{R}^p$ and dissimilarities are $d(X_i, X_j) = \|X_i - X_j\|_2^2$

Fact: within-cluster scatter can be rewritten as

$$\frac{1}{2} \sum_{k=1}^K \frac{1}{n_k} \sum_{C(i)=k} \sum_{C(j)=k} \|X_i - X_j\|_2^2 = \sum_{k=1}^K \sum_{C(i)=k} \|X_i - \bar{X}_k\|_2^2$$

with $\bar{X}_k$ the average of points in group k , $\bar{X}_k = \frac{1}{n_k} \sum_{C(i)=k} X_i$.
The right-hand side above is called **within-cluster variation**

Hence, equivalently we seek a clustering C that minimizes the within-cluster variation (approximately so)

Rewriting the minimization

Remember: we want to choose C to minimize

$$\sum_{k=1}^K \sum_{C(i)=k} \|X_i - \bar{X}_k\|_2^2$$

Question: for any $Z_1, \dots, Z_m \in \mathbb{R}^p$, suppose that we minimize the quantity $\sum_{i=1}^m \|Z_i - c\|_2^2$ over c . What is the minimizing c ?

Rewriting the minimization

With the fact from the last slide, we can introduce new variables c_k , and note that minimizing

$$\sum_{k=1}^K \sum_{C(i)=k} \|X_i - \bar{X}_k\|_2^2$$

is the same as minimizing

$$\sum_{k=1}^K \sum_{C(i)=k} \|X_i - c_k\|_2^2,$$

over both clusterings C and $c_1, \dots, c_K \in \mathbb{R}^p$

Finally...

We want to minimize

$$\sum_{k=1}^K \sum_{C(i)=k} \|X_i - c_k\|_2^2,$$

over both clusterings C and $c_1, \dots, c_K \in \mathbb{R}^p$. It's still not clear how to do this. However, can you:

Minimize it just over C ?

Minimize it just over c_k ?

K -means algorithm

The K -means clustering algorithm approximately minimizes the enlarged criterion by alternately minimizing over C and $c_1, \dots, c_K$

We start with an initial guess for $c_1, \dots, c_K$ (e.g., pick K points at random from $X_1, \dots, X_n$), then repeat:

1. Minimize over C : for each $i = 1, \dots, n$, find the cluster center c_k closest to X_i , and let $C(i) = k$
2. Minimize over $c_1, \dots, c_K$: for each $k = 1, \dots, K$, let $c_k = \bar{X}_k$, the average of points in group k

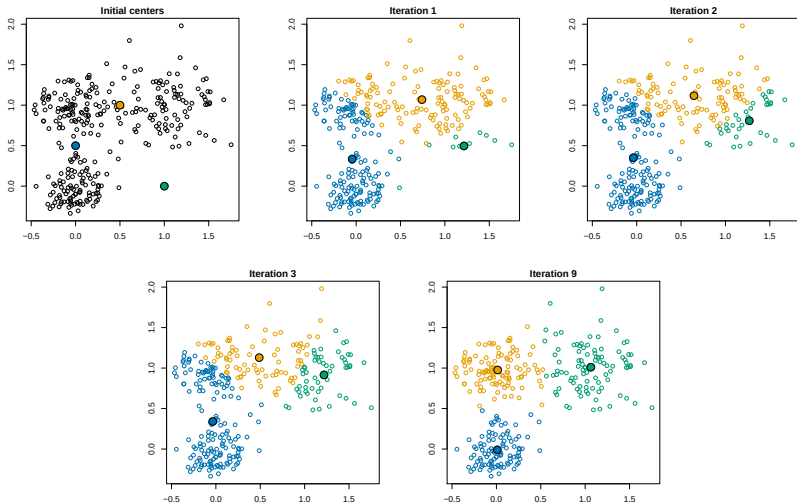
Stop when within-cluster variation doesn't change

In words:

1. Cluster (label) each point based the closest center
2. Replace each center by the average of points in its cluster

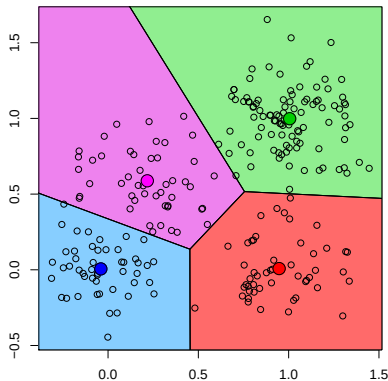
K -means example

Here $X_i \in \mathbb{R}^2$, $n = 300$, and $K = 3$



Voronoi tessellation

Given cluster centers, we identify each point to its nearest center. This defines a **Voronoi tessellation** of $\mathbb{R}^p$



Given $c_1, \dots, c_K \in \mathbb{R}^p$, we define the Voronoi sets

$$V_k = \{x \in \mathbb{R}^p : \|x - c_k\|_2^2 \leq \|x - c_j\|_2^2, j = 1, \dots, K\}, \quad k = 1, \dots, K$$

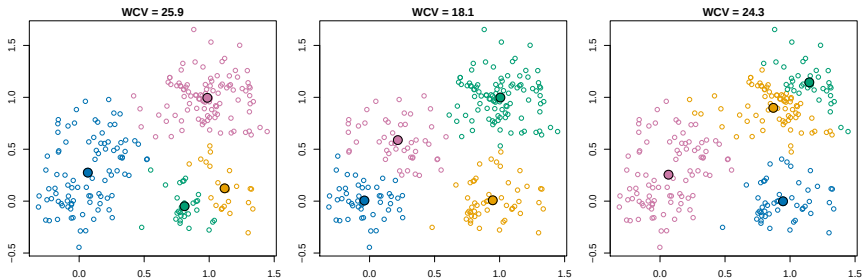
These are **convex polyhedra** (should remind you of LDA)

Properties of K -means

- ▶ Within-cluster variation **decreases** with each iteration of the algorithm. I.e., if W_t is the within-cluster variation at iteration t , then $W_{t+1} \leq W_t$
- ▶ The algorithm **always converges**, no matter the initial cluster centers. In fact, it takes $\leq K^n$ iterations (why?)
- ▶ The final clustering **depends on the initial** cluster centers. Sometimes, different initial centers lead to very different final outputs. So we typically run K -means **multiple times** (e.g., 10 times), randomly initializing cluster centers for each run, then choose among from collection of centers based on which one gives the smallest within-cluster variation
- ▶ The algorithm is **not guaranteed** to deliver the clustering that globally minimizes within-cluster variation (recall: this would require looking through all possible assignments!)

K -means example, multiple runs

Here $X_i \in \mathbb{R}^2$, $n = 250$, and $K = 4$, the points are not as well-separated



These are results of result of running the K -means algorithm with different initial centers (chosen randomly over the range of the X_i 's). We choose the second collection of centers because it yields the **smallest within-cluster variation**

What are some things K -means lacks?

K -means is a famous, standard clustering algorithm. However, it lacks several potentially-desirable qualities:

- ▶ Ability to use other measures of dissimilarity
- ▶ “Interpretable” cluster centers
- ▶ Deterministic results
- ▶ Multi-level/scale view of clusters, Nested clusters.

In K -means, cluster centers are averages

A cluster center is representative for all points in a cluster, also called a **prototype**

In K -means, we simply take a cluster center to be the **average** of points in the cluster. Great for computational purposes—but how does it lend to **interpretation**?

Sometimes we prefer methods that return a **representative** item for the cluster, rather than an average. For example: a “typical” asset or company that is similar to all the other members of the cluster. This makes it easier to think about what the cluster means.

Suppose we were clustering documents. What does an “average” document mean? A typical document is more useful.

K -medoids algorithm

K -medoids clustering addresses the first two concerns. It make each center **one of the cluster points**. It also allows other dimilarities to be substituted.

Initial guess for centers $c_1, \dots, c_K$ (e.g., randomly select K of the points $X_1, \dots, X_n$), then repeat:

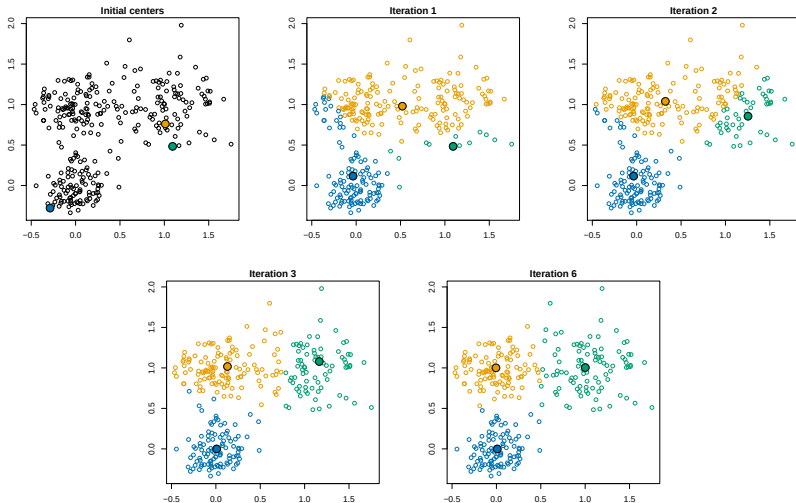
1. **Minimize over C** : for each $i = 1, \dots, n$, find the cluster center c_k closest to X_i , and let $C(i) = k$
2. **Minimize over $c_1, \dots, c_K$** : for each $k = 1, \dots, K$, let $c_k = X_k^*$, the **medoid** of points in cluster k , i.e., the point X_i in cluster k that minimizes $\sum_{C(j)=k} \|X_j - X_i\|_2^2$

Stop when within-cluster variation doesn't change

In words:

1. Cluster (label) each point based on the closest center
2. Replace each center by the medoid of points in its cluster

K -medoids example



Note: only 3 points had different labels under K -means

Properties of K -medoids

The K -medoids algorithm **shares the properties** of K -means that we discussed (each iteration decreases the criterion; the algorithm always converges; different starts gives different final answers; it does not achieve the global minimum)

K -medoids returns centers that are actual data points.

K -medoids generally returns a **higher value** of $\sum_{k=1}^K \sum_{C(i)=k} \|X_i - c_k\|_2^2$ than does K -means (why?).

K -medoids is **computationally harder** than K -means (because of step 2: computing the medoid is harder than computing the average)